

Mujin Documentation System Engineer Challenge

Welcome to your final interview at Mujin Robotics! This will be a tough challenge for you to complete remotely within 48 hours. Your task is to design, implement, test, document a simple RESTful API.

Overview

Time is limited. Please make sure you plan ahead for the following tasks:

- Set up a git repository on GitHub (or other git platform) for this project, and make sure your source code is tracked by git
- You will be starting out with a `RAML` project to describe a set of RESTful APIs
- You will then be writing a HTTP server project to implement this set of RESTful APIs
- You will create tests covering your implementation
- You will be generating documentations, both HTML and PDF, for your RESTful APIs
- Finally, you will be deploying continuous integration for your project using your git repository

Try to complete as many of the requirements as possible.

Data Models

In this challenge, you will be building a RESTful API server that allows you to manipulate the following data models over HTTP protocol.

Robot

Each robot will have the following properties:

- `id`: `string` type, unique id of the robot, for example `robot1`
- `name`: `string` type, name of the robot, for example `My Robot`
- `links`: `Link[]` type, a list of `Link` in the robot, the robot must have at least one base link
- `joints`: `Joint[]` type, a list of `Joint` in the robot

Joint

Each joint will have the following properties:

- `id`: `string` type, unique id of the joint, for example `j1`
- `name`: `string` type, name of the joint, for example `J1`
- `parentLinkId`: `string` type, `id` of the parent link, for example `base`, read-only once joint is created
- `childLinkId`: `string` type, `id` of the parent link, for example `link1`, read-only once joint is created
- `anchor`: `number[3]` type, anchor of the joint in `Robot` frame, for example `[0, 0, 0.2]`
- `axis`: `number[3]` type, axis of the joint at anchor, for example `[0, 0, 1]`

Link

Each link will have the following properties:

- `id`: `string` type, unique id of the link, for example `link1`
- `parentLinkId`: `string | nil` type, id of the parent link, for example `base`, base link does not have a `parentLinkId` and uses `nil` to denote that, read-only once link is created
- `name`: `string` type, name of the link, for example `Link 1`
- `transform`: `number[7]` type, 3D transform of the link in respect to the `Robot`, quaternion plus translation, for example `[1, 0, 0, 0, 1, 2, 3]`
- `geometries`: `Geometry[]` type, a list of geometries contained in this link

Geometry

Each geometry will have the following properties:

- `id`: `string` type, unique id of the geometry, for example `box1`
- `name`: `string` type, name of the geometry, for example `Base Plate`
- `type`: `string` type, for this challenge, only allowed value is `box`
- `extents`: `number[3]` type, a list of 3 floating point numbers, denotes the extents of the `box`, for example `[1.0, 1.0, 1.0]`
- `transform`: `number[7]` type, 3D transform of the geometry in respect to the `Link`, quaternion plus translation, for example `[1, 0, 0, 0, 1, 2, 3]`

RESTful APIs

Using the above data models, please design and implement HTTP APIs conforming to RESTful convention. Each resource should have CRUD methods (Create, Read, Update, Delete):

- **Robot APIs**
 - `GET /api/robot` - List all robots in your collection
 - `POST /api/robot` - Create a new robot, a unique `id` should be assigned by your system, base link should automatically be created
 - `GET /api/robot/{robotid}` - Get properties of a robot
 - `PUT /api/robot/{robotid}` - Modify properties of a robot
 - `DELETE /api/robot/{robotid}` - Delete a robot from your collection
- **Joint APIs**
 - `GET /api/robot/{robotid}/joint` - List all joints in the robot
 - `GET /api/robot/{robotid}/joint/{jointid}` - Get properties of a joint
 - `PUT /api/robot/{robotid}/joint/{jointid}` - Modify properties of a joint
- **Link APIs**
 - `GET /api/robot/{robotid}/link` - List all links in the robot
 - `POST /api/robot/{robotid}/link` - Create a new link in the robot, need to specify

`parentLinkId` and automatically create `Joint` to connect the links

- `GET /api/robot/{robotid}/link/{linkid}` - Get properties of a link
- `PUT /api/robot/{robotid}/link/{linkid}` - Modify properties of a link
- `DELETE /api/robot/{robotid}/link/{linkid}` - Delete a link, need to also delete associated `Joint`, cannot delete base link

- Geometry APIs

- `GET /api/robot/{robotid}/link/{linkid}/geometry` - List all geometries in the link
- `POST /api/robot/{robotid}/link/{linkid}/geometry` - Create a new geometry in the link
- `GET /api/robot/{robotid}/link/{linkid}/geometry/{geometryid}` - Get properties of a geometry
- `PUT /api/robot/{robotid}/link/{linkid}/geometry/{geometryid}` - Modify properties of a geometry
- `DELETE /api/robot/{robotid}/link/{linkid}/geometry/{geometryid}` - Delete a geometry

Additionally,

- for all `GET` methods, your API should return `200` status code and `application/json` response on success
- for all `POST` methods, your API should return `201` status code and `application/json` response on success
- for all `PUT` methods, your API should return `202` status code and `application/json` response on success
- for all `DELETE` method, your API should return `204` status code and no response body on success

Use other HTTP status code as you deem fit in other situations.

RAML

Before you start implementing the APIs, use `RAML` to describe your APIs precisely.

Server Implementation

Consider using frameworks such as `flask` or `django` to implement your server, but feel free to choose your own frameworks.

At MUJIN, `Python` and `C++` are the most widely used languages. If you plan to use other languages, please let us know first.

Tests

Once you have the HTTP server implemented, write tests to cover all your APIs. Consider using frameworks such as `pytest` to implement your tests. Consider tools such as `pytest-cov` that helps you with code coverage. Feel free to choose your own tools and frameworks

Documentations

Using your `RAML` definition, write scripts that generate documentations for your APIs:

1. a single page HTML document
2. a PDF document

Automation!

Make use of CI systems like GitHub Action, Travis CI, GitLab Pipeline to automate your project's compilation, testing and documentation generation. Automatically deploy your generated documents to a publicly viewable web server, such as GitHub Pages. Check in any automtion related scripts and configurations.

Submission

All your results should be reproducible. Submit a link to your git repository before the 48-hour deadline. If you are unable to complete all requirements, please submit your work in progress.